Private, Self-Hosted LLM Architecture

Running a large language model entirely inside your own network — no data is ever sent to Claude, ChatGPT, or Gemini.

Why local

Renting a cloud model means mailing your data to someone else's building every time you ask a question — fine for a grocery list, not for confidential records. For sensitive or regulated data you do the opposite: you bring the AI *into your own house*. Download the model, run it on your own hardware, feed it only your own documents, and cut off any path to the internet. The question, the documents, and the answer all stay inside your walls.

The system is built in layers. Each layer has a clear job and a menu of mature, self-hostable technologies that can fill it.

Architecture at a glance

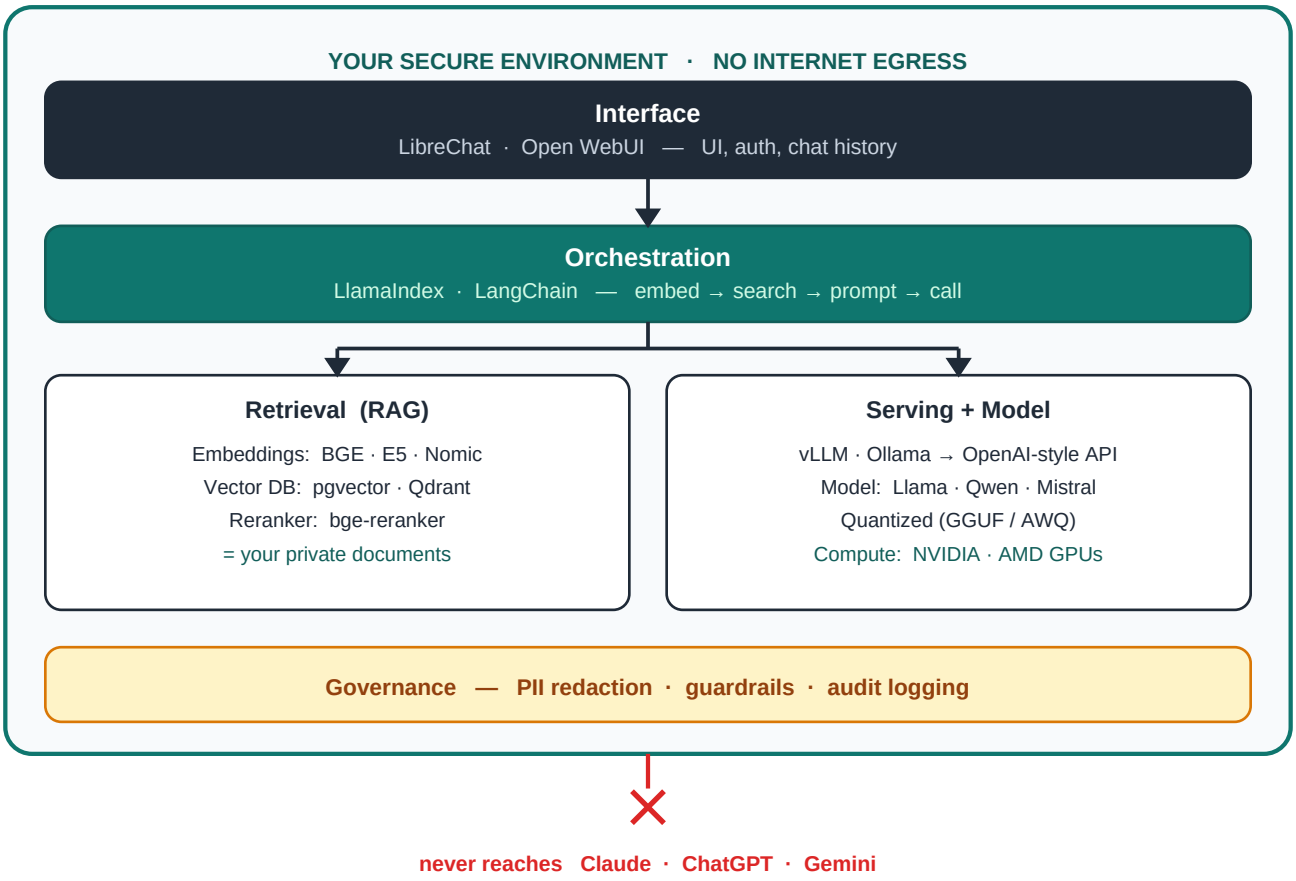

Figure 1 — The layered architecture. Every component runs inside the secure boundary; no layer has a path to the public internet.

How a request flows. A user types in the **interface** → the **orchestrator** turns the question into a search → the **retrieval** layer finds the relevant private documents → those pages plus the question go to the **inference server** running the **model** on your **GPUs** → the answer flows back, with **guardrails and audit logging** wrapping every step.

The layers and available technologies

Layer 1 Interface (Chat UI)

Where people actually type — logins, conversation history, model selection.

- **LibreChat** — self-hosted, ChatGPT-style UI; endpoint-agnostic, multi-user auth, built-in RAG (file upload + vector storage). Point it at a local serving endpoint and the whole interface runs inside your walls.
- **Open WebUI** — popular Ollama-friendly UI with RAG and role-based access.
- **AnythingLLM** — all-in-one workspace UI with built-in document ingestion.
- **Custom React / Next.js**, **Chatbot UI**, or **Streamlit / Gradio** for prototypes.

Confidentiality gotcha. UIs like LibreChat can also list cloud providers. For a sealed deployment, configure *only* local endpoints — no cloud API keys, and disable cloud endpoints in config (e.g. `librechat.yaml`) — so no one can accidentally pick a cloud model and ship data out.

Layer 2 Orchestration (the "glue")

The application logic that runs the dance: embed → search → build prompt → call the model → post-process. Also where agents and tool-calling live.

- **LlamaIndex** — retrieval-first framework, strong for RAG pipelines.
- **LangChain / LangGraph** — broad framework; LangGraph for multi-step agent flows.
- **Haystack** — production-oriented RAG and search pipelines.
- **DSPy** — programmatic prompt optimization.
- **Semantic Kernel**, or plain **Python / FastAPI** when you want full control.

Layer 3 Retrieval (RAG)

Keeps your private documents searchable by meaning, so the model reads only the relevant pages instead of memorizing everything.

- **Document ingestion & chunking** — Unstructured, LlamaParse, Apache Tika, PyMuPDF / pdfplumber.
- **Embedding models (local)** — BGE (BAAI), E5 (intfloat), Nomic-embed, GTE, Instructor, or anything via sentence-transformers.
- **Vector database** — pgvector (Postgres), Qdrant, Weaviate, Milvus, Chroma, LanceDB, FAISS (library), or Elasticsearch / OpenSearch for hybrid search.
- **Reranking (optional, high-impact)** — bge-reranker, local cross-encoders, or ColBERT to reorder retrieved pages before they hit the model.

Layer 4 Inference / Model serving

Loads the model into GPU memory and exposes a private API — usually OpenAI-compatible, so the rest of the stack talks to it like the cloud.

- **vLLM** — high-throughput production serving (PagedAttention, continuous batching).
- **TGI** (Hugging Face Text Generation Inference) — production serving with strong ops tooling.
- **SGLang** — fast serving with strong structured-output support.
- **TensorRT-LLM** — maximum performance on NVIDIA hardware.
- **Ollama** — easiest path for dev, desktop, and small deployments.
- **llama.cpp / LM Studio** — CPU and edge inference using GGUF.
- **Triton Inference Server** — when standardizing many models under one server.

Layer 5 The model (open-weight LLM)

The brain. Choose an open-weight model so the file lives on your disk; pick size by capability versus hardware budget.

- **Model families** — Llama (Meta), Qwen (Alibaba), Mistral / Mixtral, Gemma (Google), DeepSeek, Phi (Microsoft), Command R (Cohere). *The landscape moves quickly — check current open-model leaderboards for the best quality/size fit when you deploy.*
- **Quantization formats** (shrink the model to fit smaller GPUs for a small accuracy trade) — GGUF, GPTQ, AWQ, bitsandbytes (4-/8-bit).

Layer 6 Compute / Hardware

The muscle. GPU memory (VRAM) is usually the binding constraint.

- **NVIDIA data-center GPUs** — H100, A100, L40S, L4.
- **NVIDIA workstation / consumer** — RTX 6000 Ada, RTX 4090 / 5090.
- **AMD** — Instinct MI300X. **Apple Silicon (M-series)** — viable for small local / edge models.
- Hosted on **on-prem servers** or **isolated private-cloud GPU instances** with no public egress.
- *Rough sizing:* a quantized mid-size model fits on a single 24–48 GB card; large models need multiple GPUs with tensor parallelism.

Layer 7 Security & Governance (cross-cutting — the "walls")

Enforces the confidentiality guarantee and produces compliance evidence. This layer wraps all the others.

- **Network isolation** — air-gapped or VPC deployment with no outbound internet, private subnets, locked egress.
- **PII detection & redaction** — Microsoft Presidio, spaCy-based pipelines.
- **Guardrails / safety** — NVIDIA NeMo Guardrails, Guardrails AI, Llama Guard, prompt-injection filters.
- **Gateway / access control** — LiteLLM, Portkey, Kong / APISIX for auth, rate limiting, and routing — kept fully local.
- **Observability & audit** — Langfuse, Arize Phoenix, OpenLLMetry, feeding your existing SIEM for retained logs.

Layer 8 Customization (optional)

Permanently teach the model your jargon and style — usually a later step, after RAG is working.

- **LoRA / QLoRA** via PEFT, Axolotl, Unsloth, LLaMA-Factory, or TRL. Train on your own data, on your own hardware.

A reference stack

A sensible default for a regulated or confidential deployment:

Layer	Recommended choice
Interface	LibreChat (cloud endpoints disabled)
Orchestration	LlamaIndex, or LibreChat's built-in RAG
Retrieval	pgvector + BGE-large embeddings + bge-reranker
Serving	vLLM, exposing an OpenAI-compatible endpoint
Model	Mid-size open-weight model (Llama / Qwen), AWQ-quantized
Hardware	1–2× 48 GB NVIDIA GPUs, on-prem
Governance	Presidio (PII) + LiteLLM gateway + Langfuse audit logging

Key principles

- **Nothing leaves the walls** — no cloud endpoints configured, no outbound internet path.
- **RAG before fine-tuning** — cheaper, stays current, and updates without retraining the model.
- **VRAM drives cost** — size the model and its quantization to the GPUs you can afford.
- **Log everything** — audit trails are a compliance requirement, not an afterthought.